```
using CustomProjectRPG.Interfaces;
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Threading.Tasks;
using Enum = CustomProjectRPG.Entities.Enum.Enum;

namespace CustomProjectRPG.Manager
{
    public class QuestManager : ISerializeJSON<QuestManager>, IQuestManager
    {
        private Dictionary<string, QuestData> _quests; // Quest ID -> QuestData
        private ICorruption _corruption; // For favor-based quest adjustments

        public event Action<string, bool> QuestUpdated; // Event for quest status changes (questId, isComplete)


        /// Initializes a new QuestManager with a corruption reference.

        public QuestManager(ICorruption corruption)
        {
            _corruption = corruption ?? throw new ArgumentNullException(nameof(corruption));
            _quests = new Dictionary<string, QuestData>();
        }


        /// Starts a new quest with the specified type and ID.

        public void StartQuest(CustomProjectRPG.Entities.Enum.Enum.QuestType type, string questId)
        {
            if (string.IsNullOrEmpty(questId)) throw new ArgumentException("Quest ID cannot be null or empty.", nameof(questId));
            if (_quests.ContainsKey(questId)) throw new ArgumentException("Quest already exists.", nameof(questId));

            _quests[questId] = new QuestData
            {
                Type = type,
                IsActive = true,
```

```
41                  Progress = 0,
42                  RequiredProgress = GetRequiredProgress(type),
43                  IsComplete = false
44              };
45              QuestUpdated?.Invoke(questId, false); // Notify quest start
46          }
47
48
49          /// Updates the progress of a quest.
50
51          public void UpdateQuestProgress(string questId, int amount)
52          {
53              if (!_quests.ContainsKey(questId) || !_quests
                  [questId].IsActive) return;
54              if (amount < 0) throw new ArgumentException("Progress amount
                  must be non-negative.", nameof(amount));
55
56              var quest = _quests[questId];
57              quest.Progress = Math.Min(quest.Progress + amount,
                  quest.RequiredProgress);
58              quest.IsComplete = quest.Progress >= quest.RequiredProgress;
59              QuestUpdated?.Invoke(questId, quest.IsComplete);
60
61              if (quest.IsComplete)
62              {
63                  HandleQuestCompletion(questId);
64              }
65          }
66
67
68          /// Checks if a quest is complete.
69
70          public bool IsQuestComplete(string questId)
71          {
72              return _quests.ContainsKey(questId) && _quests
                  [questId].IsComplete;
73          }
74
75
76          /// Checks the favor result for a favor-based quest.
77
78          public Enum.FavorResult CheckFavorResult(string questId)
79          {
80              if (!_quests.ContainsKey(questId) || _quests[questId].Type !=
                  CustomProjectRPG.Entities.Enum.Enum.QuestType.Favor)
81              {
82                  return
                      CustomProjectRPG.Entities.Enum.Enum.FavorResult.Invalid;
83              }
```

```
84
85              var quest = _quests[questId];
86              if (quest.IsComplete)
87              {
88                  return
                      CustomProjectRPG.Entities.Enum.Enum.FavorResult.Success;
89              }
90              float favorThreshold = 50f + (_corruption.
                  CorruptionPercentage * 0.5f); // Adjust based on corruption
91              return quest.Progress >= favorThreshold ?
                  CustomProjectRPG.Entities.Enum.Enum.FavorResult.Success :
                  CustomProjectRPG.Entities.Enum.Enum.FavorResult.Insufficient
                  ;
92          }
93
94
95          /// Handles logic after quest completion (e.g., rewards, dialogue
              updates).
96
97          private void HandleQuestCompletion(string questId)
98          {
99              var quest = _quests[questId];
100             quest.IsActive = false; // Mark as inactive after completion
101                                     // Example: Trigger dialogue update
                                          via DialogueManager (assumes
                                          integration)
102                                     // dialogueManager.ShowDialogue
                                          (questId, "completed");
103         }
104
105
106         /// Determines the required progress based on quest type.
107
108         private int GetRequiredProgress
              (CustomProjectRPG.Entities.Enum.Enum.QuestType type)
109         {
110             return type switch
111             {
112                 CustomProjectRPG.Entities.Enum.Enum.QuestType.Daily => 5,
113                 CustomProjectRPG.Entities.Enum.Enum.QuestType.Story => 10,
114                 CustomProjectRPG.Entities.Enum.Enum.QuestType.Favor => 8,
115                 _ => 1
116             };
117         }
118
119
120         /// Serializes the quest state to JSON.
121
122         public string Serialize()
```

```
123         {
124             var data = new QuestManagerData
125             {
126                 Quests = _quests
127             };
128             return JsonSerializer.Serialize(data);
129         }
130 
131 
132         /// Deserializes the quest state from JSON.
133 
134         public void Deserialize(string json)
135         {
136             var data = JsonSerializer.Deserialize<QuestManagerData>(json)
137                 ?? throw new JsonException("Invalid JSON format for quest
                      state.");
138             _quests = data.Quests ?? new Dictionary<string, QuestData>();
139         }
140 
141         private class QuestData
142         {
143             [JsonPropertyName("type")]
144             public CustomProjectRPG.Entities.Enum.Enum.QuestType Type
                  { get; set; }
145 
146             [JsonPropertyName("isActive")]
147             public bool IsActive { get; set; }
148 
149             [JsonPropertyName("progress")]
150             public int Progress { get; set; }
151 
152             [JsonPropertyName("requiredProgress")]
153             public int RequiredProgress { get; set; }
154 
155             [JsonPropertyName("isComplete")]
156             public bool IsComplete { get; set; }
157         }
158 
159         private class QuestManagerData
160         {
161             [JsonPropertyName("quests")]
162             public Dictionary<string, QuestData> Quests { get; set; }
163         }
164     }
165 }
166 
```